

Analyst to Architect, Book 4 — Chapter 71,

section 71.11

Review copy, 2 October 2026. The new predict-the-output section of the SQL question bank: 16 core questions and 16 rapid-fire rows. Every answer was produced by running the query against the chapter's own `riverstone_2025` data. The confirming run on PostgreSQL 16 and MySQL 8.4 has not happened, so outputs are labelled *Run (riverstone_2025)*; the four questions marked *Dialect split* give each engine's documented behaviour instead of one result.

71.11 Predict the output: from basic to brain-racking

The hardest SQL round is not "write a query." It is a six-line query on the screen and one question: **what does this return?** You cannot hedge, you cannot talk around it, and the interviewer can see whether you know the rules or have been getting away with guessing.

Almost every question in this section turns on one of three things. SQL's NULL is not a value and does not behave like one. A join can quietly change how many rows you are aggregating. And a window function has a frame, whether or not you wrote one. Learn those three and the rest is arithmetic.

Work through these the way the round actually goes: read the query, say your answer out loud, *then* read on. An answer you guessed and an answer you reasoned to feel identical on the page and nothing like each other in the room.

How these were run. Every output below came from running the query against `riverstone_2025`, the same 24-customer database the rest of this chapter uses, so the figures agree with the numbers already printed earlier in the chapter. Four questions are marked **Dialect split**: PostgreSQL and MySQL genuinely disagree about them, and those four give each engine's documented behaviour rather than one printed result. Where a result depends on something the standard leaves to the engine, the question says so instead of pretending there is one right answer.

One reminder, because it is the single most useful fact in this section: in SQL, `NULL` means *unknown*. Any comparison with an unknown is itself unknown, and `WHERE` keeps a row only when its condition is **true** — not when it is unknown. Nearly every surprise below is that one sentence, wearing a different hat.

Q71-077 • `count(*)`, `count(city)` and `count(DISTINCT city)` on the same column: what are the three numbers? • Warm-up

Remember it as: `count(*)` counts rows. `count(col)` counts values that are not `NULL`. `count(DISTINCT col)` counts different values.

Answer in one line: 24, 24 and 17: there are 24 customers, all 24 have a city recorded, but those 24 customers sit in only 17 different cities.

```
SELECT count(*) AS star, count(city) AS col, count(DISTINCT city) AS distinct_col
FROM customers;
```

Run (riverstone_2025):

```
star | col | distinct_col
-----+-----+-----
    24 |  24 |           17
(1 row)
```

The gap between 24 and 17 is the one people miss. `count(DISTINCT city)` is not "how many customers have a city" — it is "how many cities appear at all." Mumbai and Pune each have three customers, and Ahmedabad, Bengaluru and Delhi two each, which is where the seven duplicates go.

The gap between `star` and `col` is zero here only because this table happens to be complete. On a column with gaps the two diverge, and that divergence is the subject of most of this section.

Tier	What to say
Passes	" <code>count(*)</code> counts rows and <code>count(col)</code> counts non-NULLs" (right rule, no numbers)
Strong	All three numbers with the reason for each, and the observation that <code>star = col</code> only tells you this particular column has no NULLs
Extra points	[+Validate] <code>count(*) - count(col)</code> is exactly the number of missing values in a column, which makes a one-line completeness check across a table · [+Business] 24 customers in 17 cities is a concentration fact a sales head would want to hear before a territory plan

Likely follow-ups: Which is faster, `count(*)` or `count(1)`? (*No difference on any current engine; the planner treats them the same.*) What does `count(DISTINCT a, b)` do? How would you count the NULLs themselves? (`count(*) - count(col)`, or `count(*) FILTER (WHERE col IS NULL)`.)

Red flag: saying `count(*)` skips NULL rows. It counts rows; it never looks inside them.

Learn it in: Chapter 12, section 12.9 (aggregate functions); Chapter 12, section 12.7 (NULL).

Q71-078 • `SELECT sum(quantity) FROM order_items WHERE quantity > 100000` : **what does it return? • Warm-up**

Remember it as: *sum of nothing is NULL, not zero. count of nothing is zero. The two aggregates disagree about emptiness.*

Answer in one line: `NULL`, not `0`: no row matches, and `sum` over an empty set has nothing to add up, so it returns `NULL` — while `count(*)` over the same empty set returns `0`.

```
SELECT sum(quantity) AS total, count(*) AS rows_seen
FROM order_items
WHERE quantity > 100000;
```

Run (riverstone_2025):

```

total | rows_seen
-----+-----
      |          0
(1 row)

```

That blank under `total` is a NULL. The largest quantity in the table is nowhere near 100,000, so nothing matches and the sum is undefined rather than empty-and-therefore-zero.

This is the bug that quietly breaks dashboards. A revenue tile for a region with no sales this month shows "—" instead of "₹0", or a calculation downstream touches the NULL and the whole figure disappears. The fix is one function:

```

SELECT coalesce(sum(quantity), 0) AS total
FROM order_items
WHERE quantity > 100000;

```

```

total
-----
      0
(1 row)

```

`avg` behaves the same way: over zero rows it is NULL, and there is no sensible number it could return instead.

Tier	What to say
Passes	"It returns NULL"
Strong	NULL for <code>sum</code> and <code>avg</code> , 0 for <code>count</code> , with the reason: an empty set has no total but it does have a row count of zero. Then <code>coalesce(sum(...), 0)</code> as the fix
Extra points	[+Edge cases] <code>sum</code> also returns NULL when rows <i>do</i> match but every matching value is NULL, which is a different cause with the same symptom · [+Business] wrap <code>sum</code> in <code>coalesce</code> in anything feeding a report, so an empty month reads ₹0 rather than blank · [+Validate] the two columns above prove the behaviour side by side in one query

Likely follow-ups: What does `avg` return over no rows? Does `count(col)` return 0 or NULL? (0.) How would a NULL here affect a later `WHERE total > 0`? (*The row is dropped: unknown is not true.*)

Red flag: asserting `sum` returns 0 over an empty set. It is the most common wrong answer to this question and it is a real production bug.

Learn it in: Chapter 12, section 12.9 (aggregates); Chapter 12, section 12.7 (NULL); Chapter 14, section 14.3 (missing values).

Q71-079 · 175 orders, 46 of them rep 3. Why does `WHERE sales_rep_id <> 3` return 118 and not 129? · Core

Remember it as: `<>` keeps only rows it can prove are different. An unknown rep is not provably different from 3, so it is dropped — by both `= 3` and `<> 3`.

Answer in one line: 11 orders have no sales rep recorded, and `NULL <> 3` is unknown rather than true, so those 11 rows fail both tests: $46 + 118 = 164$, not 175, and the 11 unassigned orders fall through the gap between the two filters.

```
SELECT count(*)                AS all_orders,
       count(*) FILTER (WHERE sales_rep_id = 3) AS rep_3,
       count(*) FILTER (WHERE sales_rep_id <> 3) AS not_rep_3
FROM orders;
```

Run (riverstone_2025):

all_orders	rep_3	not_rep_3
175	46	118

(1 row)

Do the arithmetic the way a reviewer would. $175 - 46 = 129$. The query says **118**. The missing 11 are the orders whose `sales_rep_id` is `NULL` — the same 11 unassigned orders behind the `NOT IN` trap in Q71-002.

This is the most dangerous question in the section, because nothing fails. You get a number, it looks plausible, and it is wrong. An analyst asked "how many orders did someone other than rep 3 handle?" answers 118 and under-reports by 11 — and if the same report also counts rep 3's 46, the two figures do not reconcile to the total, which is the only clue anything is wrong.

The three ways to ask the question, and the three answers:

```
SELECT (SELECT count(*) FROM orders WHERE sales_rep_id <> 3)                AS plain,
       (SELECT count(*) FROM orders WHERE sales_rep_id <> 3 OR sales_rep_id IS NULL) AS
with_or_null,
       (SELECT count(*) FROM orders WHERE sales_rep_id IS DISTINCT FROM 3)   AS
is_distinct_from;
```

plain	with_or_null	is_distinct_from
118	129	129

(1 row)

`IS DISTINCT FROM` is the clean way to say "different, and treat unknown as different." It is one comparison instead of two, and it cannot be got subtly wrong the way an `OR` can when someone later adds an `AND` beside it.

Tier	What to say
Passes	"NULLs are excluded" (correct, no numbers, no fix)
Strong	The arithmetic — $175 - 46 = 129$ but the query says 118, so 11 rows satisfy neither filter — plus <code>IS DISTINCT FROM</code> or the explicit <code>OR ... IS NULL</code>
Extra points	[+Validate] the reconciliation itself is the test: if "is" plus "is not" does not add to the total, a nullable column is involved · [+Trade-offs] <code>IS DISTINCT FROM</code> is clearer than <code>OR ... IS NULL</code> but is spelled differently in MySQL, which uses <code><=></code> for the NULL-safe <i>equals</i> and therefore <code>NOT (a <=> b)</code> for this test · [+Clarify] ask the business whether "not rep 3" is meant to include unassigned orders at all; often the real answer is that they should be reported as their own line

Likely follow-ups: Does the same thing happen with `>` and `<`? (Yes: every comparison with *NULL* is unknown.) What about `NOT LIKE`? (Same.) How would you catch this in review? (Check that the positive and negative filters sum to the row count.)

Red flag: "SQL treats NULL as zero" or "NULL is just an empty string." Neither is true, and a candidate who believes either will write this bug repeatedly.

Learn it in: Chapter 12, section 12.7 (NULL: the value that isn't there); Chapter 12, section 12.6 (WHERE).

Q71-080 · Building a label with `||` when one piece is NULL: what comes out? · Core · Dialect split

Remember it as: Concatenating anything onto an unknown gives an unknown. One *NULL* swallows the whole string.

Answer in one line: The whole label is NULL, not the text with a gap in it: `||` propagates NULL, so a missing rep id erases the order number and the words around it too.

```
SELECT order_id,
       sales_rep_id,
       'Order ' || order_id || ' / rep ' || sales_rep_id AS label
FROM orders
WHERE sales_rep_id IS NULL
LIMIT 3;
```

Run (riverstone_2025):

```
order_id | sales_rep_id | label
-----+-----+-----
    10015 |              |
    10036 |              |
    10038 |              |
(3 rows)
```

The `label` column is empty for every row. Not "Order 10015 / rep " — nothing at all. The literal text you typed is gone, because one unknown operand makes the whole expression unknown.

The repair names the gap instead of hiding it:

```
SELECT order_id,
       'Order ' || order_id || ' / rep ' || coalesce(sales_rep_id::text, 'unassigned') AS label
FROM orders
WHERE sales_rep_id IS NULL
LIMIT 3;
```

```
order_id |          label
-----+-----
    10015 | Order 10015 / rep unassigned
    10036 | Order 10036 / rep unassigned
    10038 | Order 10038 / rep unassigned
(3 rows)
```

Note the cast. `coalesce` needs both arguments to be the same type, and `sales_rep_id` is an integer while `'unassigned'` is text.

Dialect split. The same intent gives three different-looking answers:

	Behaviour with a NULL argument
PostgreSQL <code>a b</code>	returns NULL
MySQL <code>CONCAT(a, b)</code>	returns NULL — same as PostgreSQL
MySQL <code>CONCAT_WS(sep, a, b)</code>	skips the NULL and returns the rest

So a query ported from MySQL's `CONCAT_WS` to PostgreSQL's `||` changes behaviour silently: labels that used to be partially filled become entirely empty. MySQL does not have `||` as a concatenation operator at all by default — it reads `||` as logical OR unless `PIPES_AS_CONCAT` is set, which is its own portability trap.

Tier	What to say
Passes	"You get NULL"
Strong	NULL for the whole expression, with the reason, plus <code>coalesce</code> and the cast it needs
Extra points	[+Edge cases] <code>CONCAT_WS</code> in MySQL skips NULLs, so the same report differs by engine · [+Business] a NULL label in a dropdown or an export shows as a blank row that nobody can click or trace, which is worse than the word "unassigned" · [+Trade-offs] <code>coalesce</code> at the point of display, not in the stored data: the fact that the rep is unknown is real and should not be overwritten

Likely follow-ups: Does `CONCAT` behave differently from `||` in PostgreSQL? (Yes — PostgreSQL's `CONCAT` ignores NULLs, unlike its `||`.) What about `format()`? How would you find every such column before a release?

Red flag: expecting the literal text to survive. It does not, and a candidate who assumes it does has not watched this happen.

Learn it in: Chapter 12, section 12.8 (transforming values); Chapter 12, section 12.7 (NULL); Chapter 12, section 12.16 (the same SQL in MySQL).

Q71-081 • `avg(sales_rep_id)` and `sum(sales_rep_id) / count(*)` on the same column: why do they differ? • Core

Remember it as: `avg` divides by the count of values, not the count of rows. Hand-rolling it with `count(*)` changes the denominator.

Answer in one line: `avg` ignores NULLs on both the top and the bottom, so it divides 674 by 164; dividing by `count(*)` divides the same 674 by 175 and gives a smaller, meaningless number.

```
SELECT avg(sales_rep_id)      AS avg_builtin,  
       sum(sales_rep_id) / count(*) AS divided_by_all_rows,  
       count(*)              AS all_rows,  
       count(sales_rep_id)    AS non_null  
FROM orders;
```

Run (riverstone_2025):

```
avg_builtin | divided_by_all_rows | all_rows | non_null  
-----+-----+-----+-----  
4.1097560975609756 | 3.8514285714285714 |      175 |      164  
(1 row)
```

$674 \div 164 = 4.1098$. $674 \div 175 = 3.8514$. Same numerator, different denominator, and only the first is the average of the values that exist.

Averaging a rep id is nonsense as a business figure — it is used here because it is this database's nullable numeric column, and the arithmetic is the point. The same mechanism decides a real one: the average order value across orders where some values are missing, or an average score where some candidates were not scored. Whether the denominator should be 164 or 175 is a *business* question, not a SQL one, and the two numbers differ by 6%.

Tier	What to say
Passes	" <code>avg</code> ignores NULLs"
Strong	The two denominators, 164 and 175, named explicitly, and the point that <code>avg</code> chose one of them for you
Extra points	[+Clarify] ask which denominator the business means: "average across orders that have a rep" and "average across all orders, counting unassigned as zero" are different questions with different answers · [+Validate] print <code>count(*)</code> and <code>count(col)</code> beside any average over a column that might have gaps · [+Edge cases] if every value is NULL, <code>avg</code> is NULL, not 0 and not an error

Likely follow-ups: How would you get an average that treats missing as zero? (`sum(col) / count(*)`, or `avg(coalesce(col, 0))`.) Which is correct? (*Whichever the business asked for — say so.*) Does `avg` of an empty set error?

Red flag: reporting an average over a column with gaps without knowing the gaps are there.

Learn it in: Chapter 12, section 12.9 (aggregates); Chapter 14, section 14.3 (missing values).

Q71-082 · The same LEFT JOIN, filtered in WHERE and then in ON : 1 row or 24?

· Core

Remember it as: ON decides what matches. WHERE decides what survives. A WHERE on the right-hand table deletes the unmatched rows a LEFT JOIN just went to the trouble of keeping.

Answer in one line: 1 and 24: putting the status test in WHERE throws away every row where the join found no pending order, which turns the LEFT JOIN into an inner join, while putting it in ON keeps all 24 customers and simply shows no order for the 23 without one.

```
-- version A: the filter in WHERE
SELECT count(*) AS rows_kept
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'Pending';
```

```
rows_kept
-----
          1
(1 row)
```

```
-- version B: the same filter, moved into ON
SELECT count(*) AS rows_kept
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id AND o.status = 'Pending';
```

```
rows_kept
-----
         24
(1 row)
```

One pending order exists in the whole database, so version A returns a single row and version B returns all 24 customers with that one order attached to its customer and nothing attached to the other 23.

The mechanism is order of operations. The join runs first and produces the matched rows plus a NULL-filled row for every customer with no pending order. Then WHERE o.status = 'Pending' tests those NULL-filled rows: NULL = 'Pending' is unknown, not true, so they are all discarded. The LEFT JOIN did its job and the WHERE undid it.

Which one you want depends entirely on the question. "How many pending orders are there?" wants A. "Show me every customer and their pending order, if any" wants B — and B is the one people write A for by mistake.

There is exactly one condition on the right table that belongs in WHERE : IS NULL on its key, which is how you deliberately ask for the non-matches. That is the anti-join in Q71-001, and it works precisely *because* it tests for the NULL-filled rows instead of against them.

Tier	What to say
Passes	"WHERE can turn a LEFT JOIN into an inner join"
Strong	Both counts, the order of operations that explains them, and which question each version answers
Extra points	[+Edge cases] the one legitimate right-table WHERE is IS NULL on its key, the anti-join · [+Validate] a LEFT JOIN whose result has fewer rows than the left table is the symptom; compare against count(*) from the left table alone · [+Business] version A answers a volume question and version B answers a coverage question; a "customers with no pending order" follow-up needs B plus IS NULL

Likely follow-ups: What if the filter is on the left table instead? (*Then WHERE and ON agree, for a LEFT JOIN.*) Does this apply to RIGHT and FULL joins? (*Yes, mirrored.*) Rewrite B's intent as an aggregate per customer.

Red flag: believing ON and WHERE are interchangeable. They are, for an inner join, and that is exactly why the habit forms and then breaks on the first outer join.

Learn it in: Chapter 12, section 12.10 (JOIN); Chapter 12, section 12.11 (how the database reads your query).

Q71-083 · On a LEFT JOIN, why does the customer with no orders show a count of 1? · Core

Remember it as: count(*) counts the NULL-filled row too. count(right_table_column) does not.

Answer in one line: The LEFT JOIN manufactures one row for a customer with no match, and count(*) counts that row, so the answer is 1 instead of 0; counting a column from the right table gives the correct 0, because that column is NULL in the manufactured row.

```
SELECT c.customer_name,
       count(*)          AS wrong,
       count(o.order_id) AS right_way
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_name
HAVING count(*) <= 1
ORDER BY c.customer_name;
```

Run (riverstone_2025):

```
customer_name | wrong | right_way
-----+-----+-----
Home Plus    |     1 |         0
(1 row)
```

Home Plus is the one customer in this database who has never ordered — the same customer Q71-001 finds. The LEFT JOIN gives it one row with every orders column NULL. count(*) sees a row and says 1. count(o.order_id) looks at the value, finds NULL, and says 0.

A "orders per customer" report built with count(*) therefore reports every dormant customer as having one order. The error is small, uniform, and invisible: the number is never zero, so nobody notices that

nobody is ever at zero.

The same rule explains why `HAVING count(*) = 0` never matches anything after a `LEFT JOIN`, and why the right way to find non-matches is `WHERE o.order_id IS NULL` before any grouping.

Tier	What to say
Passes	"Use <code>count(column)</code> instead of <code>count(*)</code> "
Strong	The reason — the join manufactures a NULL-filled row and <code>count(*)</code> counts rows — plus both numbers, and which column to count (the right table's key)
Extra points	[+Validate] on a correct "activity per customer" report at least one customer should be at zero; if none is, suspect <code>count(*)</code> · [+Edge cases] <code>sum</code> of a right-table column over a non-match is NULL, not 0, which is the Q71-078 problem arriving by a different road · [+Business] dormant customers are usually the point of the report, so turning their 0 into a 1 destroys exactly the signal that was wanted

Likely follow-ups: What does `count(DISTINCT o.order_id)` give here? (*O.*) What about `sum(oi.quantity)`? (*NULL.*) How do you list only the customers at zero?

Red flag: using `count(*)` after an outer join without noticing. It is the most common silent error in reporting SQL.

Learn it in: Chapter 12, section 12.10 (JOIN); Chapter 12, section 12.9 (GROUP BY and HAVING).

Q71-084 · A self-join to show each employee's manager returns 4 rows from a 5-row table. Where did the fifth go? · Core

Remember it as: *The person at the top has no manager. An inner self-join has nothing to match her to, so it deletes the boss.*

Answer in one line: Anita Rao's `manager_id` is NULL because she runs the company, so the inner join finds no matching employee row for her and drops her — the classic way a hierarchy report loses its own root.

```
SELECT count(*) AS inner_join_rows
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id;
```

```
inner_join_rows
-----
4
(1 row)
```

Five employees in, four out. The missing row is the one the organisation chart most obviously needs.

```
SELECT e.employee_name,
       coalesce(m.employee_name, '(nobody)') AS reports_to
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id
ORDER BY e.employee_id;
```

```

employee_name | reports_to
-----+-----
Anita Rao     | (nobody)
Vikram Singh  | Anita Rao
Neha Kulkarni | Vikram Singh
Rahul Mehta   | Vikram Singh
Farah Khan    | Anita Rao
(5 rows)

```

A LEFT JOIN plus `coalesce` keeps all five and labels the root honestly. Note also that the two aliases are doing real work: `e` is the employee and `m` is the manager, and the same table is playing both parts. Q71-014 covers what happens when you forget to give them different names.

This generalises past org charts to every parent-child structure: a category tree whose top-level categories have no parent, a comment thread whose first comment replies to nothing, an account hierarchy whose holding company sits above everything. In each case the inner join silently removes the top of the tree.

Tier	What to say
Passes	"The join drops the top-level employee"
Strong	4 against 5, named — Anita Rao, <code>manager_id</code> NULL — with the LEFT JOIN plus <code>coalesce</code> fix and the two-alias point
Extra points	[+Validate] row count out must equal row count in for a report that claims to list every employee · [+Scale] this shows only one level; a full hierarchy of unknown depth needs a recursive CTE, which is Q71-029 · [+Edge cases] a cycle in the data (two employees managing each other) breaks the recursive version and needs a depth guard

Likely follow-ups: How would you show the whole chain to the top for any employee? (*Recursive CTE — Chapter 28, section 28.2.*) What if someone's manager has left and the id now points nowhere? How would you find employees with no reports?

Red flag: not noticing the count dropped. The query runs, returns rows, and looks right.

Learn it in: Chapter 12, section 12.10 (JOIN); Chapter 28, section 28.2 (recursive CTEs).

Q71-085 • GROUP BY on a column with 11 NULLs: how many groups come back? • Core

Remember it as: GROUP BY puts all the NULLs in one group together, even though no NULL equals any other NULL.

Answer in one line: Four — one for each of the three reps plus a single group holding all 11 unassigned orders, because GROUP BY treats NULLs as *not distinct from each other* even though `=` says otherwise.

```

SELECT sales_rep_id, count(*) AS orders
FROM orders
GROUP BY sales_rep_id
ORDER BY sales_rep_id;

```

Run (riverstone_2025):

sales_rep_id	orders
3	46
4	54
5	64
	11

(4 rows)

46 + 54 + 64 + 11 = 175, the full table. Unlike nearly every other question in this section, `GROUP BY` loses nothing: the NULLs get their own group rather than being discarded.

This is worth holding onto precisely because it cuts against the pattern. `WHERE` drops unknowns. `<>` drops unknowns. `NOT IN` collapses on one unknown. But `GROUP BY` gathers them up, and so do `DISTINCT` and `UNION`. The rule those three follow is "not distinct from", which is the `IS DISTINCT FROM` logic of Q71-079, not the `=` logic of everything else.

The practical consequence is that a per-rep report has a nameless fourth row. Label it rather than letting it appear as a blank:

```
SELECT coalesce(sales_rep_id::text, 'unassigned') AS rep, count(*) AS orders
FROM orders
GROUP BY sales_rep_id
HAVING count(*) > 10
ORDER BY orders DESC;
```

rep	orders
5	64
4	54
3	46
unassigned	11

(4 rows)

Grouping by `sales_rep_id` and selecting the `coalesce` of it is deliberate: group on the real column, label at the point of display.

Tier	What to say
Passes	"You get an extra row for the NULLs"
Strong	Four groups, the counts, the fact that they reconcile to 175, and the "not distinct from" rule that <code>GROUP BY</code> , <code>DISTINCT</code> and <code>UNION</code> share
Extra points	[+Validate] the group counts summing to <code>count(*)</code> proves no rows were lost, which is the check that would have caught Q71-079 · [+Business] 11 unassigned orders out of 175 is 6% of the book with no owner: a finding, not a footnote · [+Edge cases] <code>GROUP BY</code> on two columns groups on the combination, and a NULL in either still groups rather than vanishing

Likely follow-ups: Does `DISTINCT` keep one NULL or none? (*One.*) Does `UNION` deduplicate NULLs against each other? (*Yes.*) Why does `GROUP BY` disagree with `=` about NULL?

Red flag: expecting three groups. It suggests a candidate has internalised "NULLs disappear" as a universal rule rather than a property of comparison.

Learn it in: Chapter 12, section 12.9 (GROUP BY); Chapter 12, section 12.7 (NULL).

Q71-086 • ORDER BY manager_id on a column with one NULL: does the NULL come first or last? • Core • Dialect split

Remember it as: PostgreSQL sorts NULL as the largest value. MySQL sorts it as the smallest. Same query, opposite first row.

Answer in one line: It depends on the engine, and this is one of the few places where PostgreSQL and MySQL give flatly opposite answers to identical SQL: ascending, PostgreSQL puts the NULL **last** and MySQL puts it **first**.

```
SELECT employee_id, employee_name, manager_id
FROM employees
ORDER BY manager_id;
```

PostgreSQL 16 — NULLs sort as though larger than any value, so ascending puts them at the end:

employee_id	employee_name	manager_id
2	Vikram Singh	1
5	Farah Khan	1
3	Neha Kulkarni	2
4	Rahul Mehta	2
1	Anita Rao	

(5 rows)

MySQL 8.4 — NULLs sort as though smaller than any value, so ascending puts them first:

employee_id	employee_name	manager_id
1	Anita Rao	NULL
2	Vikram Singh	1
5	Farah Khan	1
3	Neha Kulkarni	2
4	Rahul Mehta	2

Both are permitted. The SQL standard leaves NULL ordering to the implementation, so neither engine is wrong and a query that relies on either is not portable.

It matters more than a tidiness point, because ORDER BY plus LIMIT is how a "top 5" is written. On the engine that sorts NULLs first, an ascending top-5 can be five rows of nothing at all.

Say what you want explicitly. PostgreSQL has the direct form:

```
SELECT employee_id, manager_id FROM employees ORDER BY manager_id NULLS FIRST;
```

MySQL has no NULLS FIRST, so the portable form — which works identically on both — sorts on a boolean first:

```
SELECT employee_id, manager_id FROM employees ORDER BY manager_id IS NULL, manager_id;
```

```
employee_id | manager_id
-----+-----
          2 |          1
          5 |          1
          3 |          2
          4 |          2
          1 |
(5 rows)
```

`manager_id IS NULL` is false (0) for the rows with a manager and true (1) for the row without, so sorting on it ascending puts the real values first on either engine. Reverse it with `IS NULL DESC` for the other arrangement.

Tier	What to say
Passes	"It depends on the database"
Strong	Which way each engine goes, that the standard permits both, and the portable <code>ORDER BY col IS NULL, col</code> form
Extra points	[+Edge cases] <code>ORDER BY</code> plus <code>LIMIT</code> on a nullable column can return an all-NULL top-N on MySQL · [+Trade-offs] <code>NULLS FIRST / NULLS LAST</code> is clearer where you only target PostgreSQL; the boolean form is the one to use in anything that has to run on both · [+Validate] an index on the column may or may not be usable depending on the null ordering you ask for, which is worth an <code>EXPLAIN</code> on a large table

Likely follow-ups: What about `DESC`? (*Each engine reverses, so PostgreSQL puts NULLs first and MySQL last.*) Does `NULLS FIRST` work in MySQL? (*No.*) How does this interact with an index?

Red flag: asserting one answer confidently without naming an engine. The whole point of the question is that there is no single answer.

Learn it in: Chapter 12, section 12.5 (`ORDER BY` and `LIMIT`); Chapter 12, section 12.16 and Chapter 13, section 13.9 (the same SQL in MySQL).

Q71-087 · A running total over a column with ties: why does it stall at 68 for three months in a row? · *Brain-racking*

Remember it as: A window with an `ORDER BY` and no frame gets `RANGE`, which treats tied rows as one. Ties share a total. Write `ROWS` when you want one row at a time.

Answer in one line: The default frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, and under `RANGE` "current row" means *every row tied with the current row*, so the three months that all have 13 orders are treated as a single step and all three report the same cumulative 68.

This is the question that separates people who have read about window functions from people who have debugged one.

```

WITH monthly AS (
  SELECT date_trunc('month', order_date)::date AS month, count(*) AS orders
  FROM orders
  GROUP BY 1
)
SELECT to_char(month, 'YYYY-MM') AS month,
       orders,
       sum(orders) OVER (ORDER BY orders) AS running
FROM monthly
ORDER BY orders, month;

```

Run (riverstone_2025):

month	orders	running
2025-01	8	8
2025-02	10	18
2025-03	11	29
2025-04	13	68
2025-06	13	68
2025-07	13	68
2025-08	14	82
2025-05	15	97
2025-09	18	133
2025-12	18	133
2025-10	21	175
2025-11	21	175

(12 rows)

Look at what happens at every tie. Three months have 13 orders, and all three show 68 — which is $29 + 13 + 13 + 13$, the total *through the end of the tie group*, not through each row. The same at 18 (September and December both show 133) and at 21 (October and November both show 175). The column jumps $29 \rightarrow 68$ in one step and never shows 42 or 55 at all.

Nothing is broken. `RANGE` works on *values*, not positions: the frame is "every row whose `orders` value is less than or equal to mine", and all three 13s qualify for each other. Asking for `ROWS` instead changes the question to "every row up to and including my position":

```

sum(orders) OVER (ORDER BY orders
                  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running

```

month	orders	running
2025-01	8	8
2025-02	10	18
2025-03	11	29
2025-04	13	42
2025-06	13	55
2025-07	13	68
2025-08	14	82
2025-05	15	97
2025-09	18	115
2025-12	18	133
2025-10	21	154
2025-11	21	175

(12 rows)

Now it increments one row at a time: 42, 55, 68. Both columns end at 175, because both eventually include everything — the disagreement is entirely about what happens *in the middle*, which is exactly where a running total is read.

One more turn of the screw. Under `ROWS`, *which* of the three tied months gets 42 and which gets 68 is not determined by anything you wrote. The engine may order tied rows however it likes, so that assignment can change between runs, between versions, and between engines. If you want `ROWS` and you have ties, you must break the tie inside the window:

```
sum(orders) OVER (ORDER BY orders, month
                  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running
```

With `month` as a tiebreak the result above is reproducible — and note that adding the tiebreak also makes `RANGE` and `ROWS` agree, because once no two rows tie there are no peer groups left for them to disagree about.

The practical lesson is short. Order a running total by something unique — a date, an id — and the default frame never bites. Order it by a measure that can tie, and you must choose a frame deliberately.

Tier	What to say
Passes	"There's something about the window frame"
Strong	Names the default as <code>RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW</code> , explains that <code>RANGE</code> includes all peers of the current row, and reads the actual numbers off both versions
Extra points	[+Edge cases] under <code>ROWS</code> with ties the per-row assignment is engine-determined, so a tiebreak in the window's <code>ORDER BY</code> is required for a reproducible answer · [+Validate] both versions must end at the same grand total, 175, which is the check that the frame is the only difference · [+Trade-offs] <code>RANGE</code> is the right default for "cumulative through this value" questions such as a price percentile; <code>ROWS</code> is right for "cumulative through this row" such as a ledger balance · [+Scale] <code>RANGE</code> has to identify peer groups and is generally slower than <code>ROWS</code> on a large partition

Likely follow-ups: What is the default frame when there is no `ORDER BY` in the window? (*The whole partition — which is why `sum(x) OVER ()` gives a grand total.*) What does `RANGE BETWEEN INTERVAL '7 days' PRECEDING AND CURRENT ROW` do on a date column? Does `ROW_NUMBER` care about the frame? (*No — ranking functions ignore the frame.*)

Red flag: insisting the output is a bug. It is documented, standard behaviour, and a candidate who calls it broken will not find it in their own code.

Learn it in: Chapter 13, section 13.3 (window functions); Chapter 28, section 28.3 (advanced window frames).

Q71-088 • `ORDER BY count(*) DESC LIMIT 5` on the busiest order days: is the answer reliable? • Brain-racking

Remember it as: A `LIMIT` is only as deterministic as its `ORDER BY`. If rows tie at the cut-off, which ones you get is arbitrary.

Answer in one line: No: one day has 4 orders and **five** days have 3, so a top-5 has to pick four of those five tied days, and nothing in the query says which four — the result can differ between runs and between engines without any of them being wrong.

```
SELECT order_date, count(*) AS orders
FROM orders
GROUP BY order_date
ORDER BY count(*) DESC
LIMIT 5;
```

Run (riverstone_2025):

order_date	orders
2025-06-10	4
2025-05-04	3
2025-11-09	3
2025-07-26	3
2025-11-16	3

(5 rows)

The shape of the tie, which is what makes the answer unsafe:

```
WITH d AS (SELECT order_date, count(*) AS n FROM orders GROUP BY order_date)
SELECT n AS orders_that_day, count(*) AS how_many_days
FROM d GROUP BY n ORDER BY n DESC;
```

orders_that_day	how_many_days
4	1
3	5
2	29
1	98

(4 rows)

One day at 4, five days at 3. The top-5 takes the 4 and then four of the five 3s, leaving one out for no stated reason. Re-run it after the data changes, or run it on a different engine, and a different day may be the one omitted — while the report still says "top 5 busiest days" with a straight face.

Two ways out, and they answer different questions:

```
-- A: make it deterministic. Still 5 rows, but always the same 5.
ORDER BY count(*) DESC, order_date
LIMIT 5;
```

```
-- B: include the whole tie. May return more than 5 rows, and should.
SELECT order_date, orders FROM (
  SELECT order_date, count(*) AS orders,
    rank() OVER (ORDER BY count(*) DESC) AS rnk
  FROM orders GROUP BY order_date
) t
WHERE rnk <= 5
ORDER BY orders DESC, order_date;
```

Version A is honest but arbitrary: it picks by date because something has to pick, and at least it picks the same way every time. Version B says what a business person usually means by "the top days" — if five days tie for second place, all five are in the answer. `rank()` is the right function here precisely because it shares a rank among ties, which is Q71-034's distinction put to work.

Worth saying out loud in the interview: *this is a question about the data, not just the query*. You cannot tell whether a top-N is safe without checking whether anything ties at the boundary, which is a one-line check nobody runs.

Tier	What to say
Passes	"Ties make <code>LIMIT</code> unpredictable"
Strong	The tie structure — one day at 4, five at 3 — and both fixes, with the point that a deterministic tiebreak and including the whole tie are different answers to different questions
Extra points	[+Validate] before trusting any top-N, count how many rows sit at the boundary value · [+Business] dropping one of five equally busy days from an operations report is the kind of error that is never caught because the output always looks plausible · [+Edge cases] the same applies to <code>ROW_NUMBER() = 1</code> for "the latest row per group": with two rows at the same timestamp, which one you get is arbitrary

Likely follow-ups: How would you return exactly 5 rows but choose the tie fairly? Does `FETCH FIRST 5 ROWS WITH TIES` help? (Yes where supported — PostgreSQL 13+; MySQL has no equivalent.) What is the cost of the window-function version on a large table?

Red flag: treating the printed five rows as *the* answer. The query does not have one answer, and saying so is the whole point.

Learn it in: Chapter 12, section 12.5 (ORDER BY and LIMIT); Chapter 13, section 13.4 (RANK and DENSE_RANK).

Q71-089 • `SELECT 10 / 0` : what happens? • Brain-racking • Dialect split

Remember it as: PostgreSQL raises an error and takes the transaction with it. MySQL returns `NULL` and carries on. Same expression, different blast radius.

Answer in one line: PostgreSQL raises `division by zero` and the statement fails — inside a transaction, that aborts it — while MySQL returns `NULL` without complaint, which means the same ETL step either stops loudly or produces quiet nonsense depending on the engine.

PostgreSQL 16:

```
ERROR:  division by zero
```

MySQL 8.4:

```
+-----+
| 10 / 0 |
+-----+
|      NULL      |
+-----+
```

MySQL's behaviour follows from its default `ERROR_FOR_DIVISION_BY_ZERO` handling in `sql_mode`; with strict mode configured to raise, it errors instead. So "MySQL returns NULL" is a statement about a default, not about the engine forever — worth saying, because it shows you know the behaviour is configurable rather than fundamental.

Neither default is good news in a report. An error at least tells you something is wrong. A NULL flows downstream, lands in a percentage column, and reads as a missing figure rather than an impossible one.

The guard is the same on both engines, and it is what the interviewer is listening for:

```
SELECT round(100.0 * delivered / nullif(total, 0), 1) AS pct_delivered
FROM (
  SELECT count(*) FILTER (WHERE status = 'Delivered') AS delivered,
         count(*) AS total
  FROM orders
) t;
```

`nullif(total, 0)` turns a zero denominator into NULL *before* the division, so the result is NULL by your decision rather than by an error or an engine default. Pair it with `coalesce` if the report should read 0% rather than blank — but only if 0% is actually true, which for "percentage of nothing" it usually is not.

This is the single most common guard in production analytics SQL. A denominator that is non-zero in every test dataset becomes zero the first week a new region launches with no orders yet.

Tier	What to say
Passes	"You should guard against dividing by zero"
Strong	Both behaviours named by engine, that MySQL's depends on <code>sql_mode</code> , and <code>nullif(denominator, 0)</code> as the portable guard
Extra points	[+Edge cases] in PostgreSQL inside a transaction the error aborts everything after it, so one bad row can roll back a whole batch load · [+Business] a new region with no orders yet is the usual trigger, and it arrives in production rather than in test · [+Trade-offs] <code>nullif</code> then <code>coalesce</code> to 0 only when zero is the honest answer; "no data" and "zero percent" are different facts

Likely follow-ups: What about `0 / 0`, or modulo by zero? What does a `NUMERIC` versus `FLOAT` division do differently? (*Floating-point division by zero can give infinity rather than an error, depending on type and engine.*) How would you find every unguarded division in a codebase?

Red flag: confident that it errors everywhere, or that it returns NULL everywhere. It is exactly the sort of question where one answer marks you as having used one database and assumed it was SQL.

Learn it in: Chapter 12, section 12.8 (transforming values); Chapter 12, section 12.16 and Chapter 13, section 13.9 (MySQL differences).

Q71-090 • `SELECT 7 / 2` : does it give 3 or 3.5? • Brain-racking • Dialect split

Remember it as: PostgreSQL keeps integer division integer and gives 3. MySQL converts and gives 3.5. The decimal point moves when you change database.

Answer in one line: PostgreSQL returns **3**, because dividing an integer by an integer produces an integer and the remainder is discarded; MySQL returns **3.5000**, because it converts to a decimal first — and the same report therefore differs by engine.

PostgreSQL 16:

```
?column?
-----
      3
(1 row)
```

MySQL 8.4:

```
+-----+
| 7 / 2 |
+-----+
| 3.5000 |
+-----+
```

PostgreSQL is following the SQL standard here: the result type of `integer / integer` is integer, and 3.5 is not one, so the remainder is discarded. Note that it **truncates rather than rounds**: `7 / 2` is 3, not the 4 you would get by rounding 3.5. The two differ by one whenever there is a remainder of a half or more, so a figure computed this way is not even reliably the nearest whole number.

The fix is to make at least one operand non-integer, and there are three common spellings:

```
SELECT 7.0 / 2      AS decimal_literal,  -- 3.5
       7::numeric / 2 AS cast_numeric,   -- 3.5
       7 * 1.0 / 2  AS multiply_first;   -- 3.5
```

The one that bites in real work is a percentage:

```
SELECT count(*) FILTER (WHERE status = 'Cancelled') / count(*) AS wrong,
       100.0 * count(*) FILTER (WHERE status = 'Cancelled') / count(*) AS right_way
FROM orders;
```

PostgreSQL 16 — the integer division happens before anything else can save it:

```
wrong |      right_way
-----+-----
      0 | 1.142857142857142857
(1 row)
```

Two cancelled orders out of 175 is 1.14%. The integer version reports **0** — not "about 1%", not a rounding error, but zero, which reads as "no cancellations at all." Every rate below 50% truncates to 0 this way, so the bug is invisible on good data and catastrophic on the exact figure a manager asked for.

On MySQL the same query returns 0.0114 in the `wrong` column rather than 0, because it converts instead of truncating. That is not a reprieve: the column is then a fraction where a percentage was intended, which is wrong by a factor of a hundred rather than wrong by everything. Both engines need the `100.0`.

Put the `100.0` at the front, before the division, and the whole expression is decimal from the start.

Tier	What to say
Passes	"Integer division truncates"
Strong	3 on PostgreSQL, 3.5 on MySQL, that truncation is toward zero rather than rounding, and a fix that makes one operand decimal
Extra points	[+Business] every rate under 50% becomes 0, so a cancellation rate, a conversion rate or a margin silently reads as nothing · [+Validate] the two columns above side by side, 0 against 1.14%, on real data · [+Edge cases] <code>-7 / 2</code> is <code>-3</code> in PostgreSQL, truncating toward zero rather than toward negative infinity, which differs from Python's <code>//</code> · [+Trade-offs] <code>100.0 * first</code> rather than casting afterwards: casting the already-truncated result cannot recover the lost remainder

Likely follow-ups: How do you get the remainder? (`%` or `mod()`.) What does `-7 / 2` give, and how does that compare with Python? What is `div` in MySQL? (*Explicit integer division, the opposite problem.*)

Red flag: "SQL always returns a decimal." Half the engines do not, and the half that do not include the one most analytics runs on.

Learn it in: Chapter 12, section 12.8 (transforming values); Chapter 12, section 12.16 (the same SQL in MySQL).

Q71-091 • `sum(count(*)) OVER ()` beside a `GROUP BY` : is that even legal? • *Brain-racking*

Remember it as: *Aggregates run first, window functions run after. So a window function can take an aggregate as its input, and that is how you get a share-of-total in one pass.*

Answer in one line: Yes, it is legal and it is the idiomatic way to compute each group's share of the whole: the `GROUP BY` collapses the rows first, and the window function then runs over those already-aggregated rows, so `sum(count(*)) OVER ()` is "the total of all the group counts."

```
SELECT segment,
       count(*)           AS in_segment,
       sum(count(*)) OVER () AS all_customers,
       round(100.0 * count(*) / sum(count(*)) OVER (), 1) AS pct
FROM customers
GROUP BY segment
ORDER BY segment;
```

Run (riverstone_2025):

segment	in_segment	all_customers	pct
Hospitality	9	24	37.5
Retail	9	24	37.5
Wholesale	6	24	25.0

(3 rows)

Most candidates say this errors, and the instinct is understandable: "you can't nest aggregates" is a real rule, and `sum(count(*))` looks exactly like nesting. It is not. The two run in different phases. `count(*)` is computed as part of the `GROUP BY`, producing three rows; `sum(... ) OVER ()` is then evaluated across

those three rows, in the window phase that comes after grouping. `sum(sum(x))` would be the illegal version — two aggregates in the *same* phase.

The payoff is that a share-of-total needs no subquery, no CTE and no second scan of the table. The alternative people reach for first is strictly more work:

```
-- the same answer, the long way
WITH per_segment AS (
  SELECT segment, count(*) AS in_segment FROM customers GROUP BY segment
)
SELECT segment, in_segment,
       round(100.0 * in_segment / (SELECT sum(in_segment) FROM per_segment), 1) AS pct
FROM per_segment ORDER BY segment;
```

Note the `100.0`, for the reason Q71-090 just gave: written as `100 *` with integer inputs, every percentage here would be 0 or 1.

The empty `OVER ()` is doing specific work. No `PARTITION BY` means one partition containing every row, and no `ORDER BY` means the frame is the whole partition — so it is a grand total. Add `PARTITION BY region` and you get share-within-region instead, which is the same idiom one step up.

Tier	What to say
Passes	"I think that errors" (the common answer, and wrong)
Strong	Legal, with the phase argument: <code>GROUP BY</code> aggregates first, window functions run over the grouped rows, so this is not nesting — and reads the percentages off
Extra points	[+Scale] one pass instead of the CTE-plus-subquery version, and no second scan · [+Edge cases] <code>sum(sum(x))</code> is illegal, because both are in the aggregate phase; the legality comes entirely from <code>OVER ()</code> moving the outer one to a later phase · [+Business] share-of-total is what nearly every management report actually wants, and this is the one-line form · [+Validate] the <code>pct</code> column should sum to 100, allowing for rounding: $37.5 + 37.5 + 25.0 = 100.0$ ✓

Likely follow-ups: What does `PARTITION BY` change here? What is the default frame with no `ORDER BY`? (*The whole partition.*) Can you use a window function in `WHERE`? (*No — `WHERE` runs before the window phase; wrap it in a subquery or use `QUALIFY` where supported.*)

Red flag: "you can never put an aggregate inside another function." The rule is about phases, not about syntax, and this is the question that tests whether a candidate knows which.

Learn it in: Chapter 13, section 13.3 (window functions); Chapter 13, section 13.7 (patterns for ranking and shares); Chapter 12, section 12.11 (how the database reads your query).

Q71-092 • WHERE city NOT IN ('Mumbai', NULL) : how many rows? • Core

Remember it as: A literal `NULL` in a `NOT IN` list is the same trap as a `NULL` from a subquery. Zero rows, every time, silently.

Answer in one line: Zero — not the 21 non-Mumbai customers — because `NOT IN` requires the row to be provably different from *every* list member, and nothing is provably different from `NULL`; meanwhile plain `IN` with the same list returns the 3 Mumbai customers quite happily.

```
SELECT (SELECT count(*) FROM customers WHERE city IN ('Mumbai', NULL))      AS in_list,
       (SELECT count(*) FROM customers WHERE city NOT IN ('Mumbai', NULL)) AS not_in_list,
       (SELECT count(*) FROM customers WHERE city = 'Mumbai')              AS plain_equals;
```

Run (riverstone_2025):

```
in_list | not_in_list | plain_equals
-----+-----+-----
      3 |           0 |           3
(1 row)
```

The asymmetry is the lesson. `IN` is a chain of `OR`s: `city = 'Mumbai' OR city = NULL`. For a Mumbai customer the first test is true, and true-OR-unknown is true, so the row is kept — the `NULL` in the list is harmless. `NOT IN` is a chain of `AND`s: `city <> 'Mumbai' AND city <> NULL`. The second test is unknown for *every* row, and anything-AND-unknown is never true, so every row is rejected.

So one `NULL` in the list is invisible to `IN` and fatal to `NOT IN`. Q71-002 showed this arriving from a subquery, which is how it happens in production; this version shows it written down in plain sight, which is how it appears in an interview. Both have the same cause and the same two fixes: filter the `NULL`s out of the list, or use `NOT EXISTS`, which asks whether a matching row exists rather than comparing against a list.

A literal `NULL` in a hand-written list looks contrived until you have seen a list built by a code generator, a reporting tool's "selected values" parameter, or a string of ids pasted from a spreadsheet with one blank cell in it.

Tier	What to say
Passes	" <code>NOT IN</code> with a <code>NULL</code> returns nothing"
Strong	Zero against 3, with the <code>AND</code> -chain versus <code>OR</code> -chain explanation for why <code>NOT IN</code> breaks and <code>IN</code> does not
Extra points	[+Edge cases] the asymmetry is the memorable part: the same <code>NULL</code> is harmless in <code>IN</code> and fatal in <code>NOT IN</code> · [+Trade-offs] <code>NOT EXISTS</code> is immune by construction and is the better default · [+Validate] "is" plus "is not" failing to reconcile to the row count is the same symptom as Q71-079

Likely follow-ups: Rewrite it safely. Does `NOT IN` against an *empty* subquery return everything or nothing? (*Everything — there is nothing to fail against.*) What about `ALL` and `ANY`? (`<> ALL` has the identical problem.)

Red flag: answering 21. It is the arithmetic answer and it is the one the trap is built to extract.

Learn it in: Chapter 12, section 12.7 (`NULL`); Chapter 12, section 12.12 (subqueries and set operations).

Rapid-fire, §71.11, part A: predict the output, warm-up to core

Say the answer before you read it. Every figure below was run on `riverstone_2025`.

#	Query	What it returns, and why	Extra point
Q71-093	<code>SELECT NULL = NULL;</code>	NULL, not true: an unknown compared with an unknown is unknown. Use <code>IS NULL</code>	[+Edge cases] <code>NULL IS NULL</code> is true; it is the only test that works → Ch 12 §12.7
Q71-094	<code>SELECT count(*) FROM orders WHERE sales_rep_id = NULL;</code>	0 rows, no error: the condition is unknown for all 175 rows, including the 11 that are NULL	[+Validate] 0 where you expected 11 is the signature of <code>= NULL</code> → Ch 12 §12.7
Q71-095	<code>SELECT sum(sales_rep_id) FROM orders WHERE sales_rep_id IS NULL;</code>	NULL, with <code>count(*)</code> on the same rows giving 11: eleven rows matched, none had a value to add	[+Edge cases] <code>sum</code> is NULL both when no rows match and when no values do; the symptom hides two causes → Ch 12 §12.9
Q71-096	<code>SELECT count(CASE WHEN city = 'Mumbai' THEN 1 END) FROM customers;</code>	3, not 24: <code>CASE</code> with no <code>ELSE</code> returns NULL for non-matches, and <code>count</code> skips NULLs. This is the conditional-count idiom	[+Trade-offs] <code>count(*) FILTER (WHERE ...)</code> is clearer in PostgreSQL; <code>CASE</code> is the portable spelling → Ch 12 §12.8
Q71-097	<code>SELECT sum(CASE WHEN city = 'Mumbai' THEN 1 ELSE 0 END) FROM customers;</code>	3 as well, by a different route: <code>ELSE 0</code> makes every row contribute, and the zeros add nothing	[+Edge cases] drop the <code>ELSE 0</code> and <code>sum</code> still gives 3, since it skips the NULLs — but it returns NULL instead of 0 when nothing matches → Ch 12 §12.8
Q71-098	<code>SELECT count(*) FROM orders o JOIN order_items i ON i.order_id = o.order_id;</code>	330, not 175: each order has several lines, so joining multiplies the order rows. <code>count(DISTINCT o.order_id)</code> is still 175	[+Validate] a row count that rises after a join is fan-out; any <code>sum</code> of an order-level column is now inflated → Ch 12 §12.10
Q71-099	<code>SELECT count(DISTINCT segment) FROM customers; where one segment is NULL</code>	Counts only the real segments; <code>DISTINCT</code> in <code>count</code> skips NULL even though <code>GROUP BY</code> would give it a group. (No NULL segments in this database: the rule, not a run)	[+Edge cases] <code>count(DISTINCT col)</code> and <code>GROUP BY col</code> disagree about NULL by exactly one group → Ch 12 §12.9
Q71-100	<code>SELECT 'abc' = 'abc';</code>	False in PostgreSQL for <code>varchar</code> ; true in MySQL's default collation, which ignores trailing spaces. <code>CHAR(n)</code> pads and compares equal on both	[+Edge cases] a join key with a stray trailing space matches on one engine and not the other → Ch 12 §12.16
Q71-101	<code>SELECT count(*) FROM customers GROUP BY segment;</code>	Three rows (9, 9, 6), not one: <code>GROUP BY</code> returns one row per group, so a bare <code>count(*)</code> is a count <i>per segment</i>	[+Business] reading only the first row of a grouped result is how a segment total gets reported as a company total → Ch 12 §12.9

#	Query	What it returns, and why	Extra point
Q71-102	SELECT segment, count(*) FROM customers GROUP BY 1 ORDER BY 2 DESC;	Works: 1 and 2 are ordinal positions in the select list, not literals. Returns 9, 9 and 6 — but Hospitality and Retail both have 9, so which of the two comes first is arbitrary	[+Trade-offs] fine for ad-hoc work, poor in stored code: inserting a column silently changes the grouping · [+Edge cases] the tie makes the row order unstable, the Q71-088 problem in miniature → Ch 12 §12.14

Rapid-fire, §71.11, part B: going deeper

#	Query or question	What it returns, and why	Extra point
Q71-103	<code>sum(x) OVER ()</code> with no <code>ORDER BY</code> — what frame?	The whole partition, so every row shows the grand total. Adding an <code>ORDER BY</code> changes the default frame to <code>RANGE ... CURRENT ROW</code> and turns it into a running total	[+Edge cases] this is why adding an <code>ORDER BY</code> to a working <code>OVER ()</code> total silently changes every number → Ch 28 §28.3
Q71-104	The three months tied on 13 orders, ranked by <code>ORDER BY orders</code>	<code>ROW_NUMBER</code> gives 4, 5, 6 — it never ties, so which tied month gets which number is arbitrary. <code>RANK</code> gives 4, 4, 4 and then jumps to 7. <code>DENSE_RANK</code> gives 4, 4, 4 and then 5	[+Validate] <code>RANK</code> skips numbers after a tie and <code>DENSE_RANK</code> does not, so the two diverge from that point on: at 14 orders they read 7 and 5 → Ch 13 §13.4
Q71-105	<code>WHERE row_number() OVER (...) = 1</code>	An error: window functions are evaluated after <code>WHERE</code> . Wrap it in a subquery or CTE and filter outside	[+Edge cases] the same reason an alias from <code>SELECT</code> is unavailable in <code>WHERE</code> → Ch 12 §12.11, Q71-010
Q71-106	<code>NOT IN</code> against a subquery that returns no rows at all	Every outer row: there is nothing for the comparison to fail against, so it is vacuously true. The opposite failure mode to one <code>NULL</code>	[+Edge cases] <code>NOT IN</code> returns everything on an empty subquery and nothing on a subquery with one <code>NULL</code> → Ch 12 §12.12
Q71-107	<code>LEFT JOIN</code> then <code>WHERE right_table.key IS NULL</code>	The anti-join: the non-matching left rows only. The one right-table condition that belongs in <code>WHERE</code> rather than <code>ON</code>	[+Trade-offs] compare with <code>NOT EXISTS</code> , which expresses the same intent without relying on the <code>NULL</code> -filled row → Ch 12 §12.10, Q71-001
Q71-108	<code>SELECT count(*) FROM orders WHERE order_date BETWEEN '2025-01-01' AND '2025-01-31';</code>	On this <code>DATE</code> column, every January order — <code>BETWEEN</code> is inclusive on both ends. On a <code>TIMESTAMP</code> column the same query drops anything after midnight on the 31st	[+Edge cases] the half-open form <code>>= '2025-01-01' AND < '2025-02-01'</code> is correct for both types; Riverstone's own column is a <code>DATE</code> , which is exactly why the bug survives review elsewhere → Ch 12 §12.6, Q71-007